

Часть 1. Аналитический блиц: 5 задач за 25 минут

Задача – правильно решить как можно больше задач за отведенное время!

Задача 1. Разбор логов доступа

Дано:

Файл access.log (nginx-style). Пример:

```
192.168.1.1 - - [20/May/2025:13:45:12 +0000] "GET /index.html HTTP/1.1" 200 512
```

Задание:

Посчитать количество запросов от каждого уникального IP и вывести топ-1 по частоте.

Пример вывода:

```
192.168.1.1: 42 запроса
```

Задача 2. Анализ транзакций

Дано:

Файл sales.csv со следующей примерной структурой:

```
date,product,amount
2025-05-20,apple,150
2025-05-20,banana,80
2025-05-21,apple,200
2025-05-21,orange,120
```

Задание:

Найти **общую сумму продаж по каждому продукту** и вывести продукт с максимальной выручкой.

Пример вывода:

```
apple: 350
```

Задача 3. Поиск выбросов в числовом ряду

Дано:

Файл `numbers.txt` множество целых чисел, каждое на новой строке, например:

```
15
22
19
45
18
21
20
23
...
```

Задание:

Найти все числа, которые отличаются от среднего арифметического более чем на 2 стандартных отклонения (выбросы).

Пример вывода:

Выбросы: 45, 48, 12

Задача 4. Слияние и агрегация из двух файлов

Дано:

Файл `users.csv` со следующей примерной структурой:

```
user_id,name
1,Alice
2,Bob
3,Charlie
```

Файл `orders.csv` со следующей примерной структурой:

```
order_id,user_id,amount
101,1,500
102,2,300
103,1,200
104,3,400
```

Задание:

Посчитать сумму заказов по каждому имени пользователя и вывести результат в виде:
Alice: 700, Bob: 300, Charlie: 400

Порядок вывода – по имени (алфавитный).

Задача 5. Валидация данных по регулярному выражению

Дано:

Файл `emails.txt` содержит список email-адресов (каждый – на новой строке):

`alice@example.com`

`bob@`

`charlie@domain`

`not-an-email`

`valid@test.org`

Задание:

Проверить каждую строку на соответствие **простому шаблону email**:

`локальная_часть@домен.доменная_зона` (от 2 до 4 букв)

Вывести только валидные email-адреса.

Пример вывода:

`alice@example.com`

`valid@test.org`

Часть 2. Продукт: «Канбан-доска за 25 минут»

Концепция

Вы создаете **одностраничное приложение** – **персональную канбан-доску** для управления задачами. Три колонки: «Нужно сделать» (To Do), «В работе» (In Progress), «Готово» (Done). Пользователь может:

- Добавлять новые задачи (карточки) в первую колонку
- Перетаскивать карточки между колонками мышью
- Редактировать текст карточки двойным кликом
- Удалять карточки
- Видеть порядок карточек внутри колонки (можно менять перетаскиванием — опционально, но приветствуется)

Все изменения (текст, положение, колонка) сохраняются в `localStorage` (файл) и восстанавливаются после перезагрузки.

Важно! Это не просто «todo list с тремя списками», а полноценная имитация trello/jira, но в минимальном виде.

Детальные функциональные требования

1. Внешний вид и базовая структура

- **Три колонки**, расположенные горизонтально (flex или grid).
- У каждой колонки есть:
 - Заголовок (To Do / In Progress / Done)
 - Контейнер для карточек (например, `<div class="card-list">`)
 - Счетчик карточек в колонке (опционально, но 加分)
- Карточка:
 - Текст задачи
 - Кнопка удаления (крестик / корзина) в правом верхнем или правом нижнем углу
 - (Опционально) иконка для захвата перетаскивания (чтобы не перетаскивать по любому клику)

CSS-требования (минимальные, но чистые):

- Колонки имеют фон (серый/белый), скругленные углы, тень
- Карточки: белый фон, внутренний отступ, граница, небольшой margin-bottom
- Разные визуальные состояния при перетаскивании (полупрозрачная копия, подсветка целевой колонки)

2. Создание карточек

- **Поле ввода** + кнопка «**Добавить**» над колонками (или под заголовком первой колонки).
- При нажатии:
 - Текст не пустой (удаляем пробелы по краям)
 - Создается новая карточка с уникальным id (например, `Date.now()`)
 - Карточка добавляется в **конец** колонки «To Do»
- Поле ввода очищается
- Ограничения: максимум 50 карточек на всех колонках (чтобы не перегружать интерфейс)

3. Drag & Drop (перетаскивание между колонками)

Обязательно: карточки можно перетаскивать из любой колонки в любую другую.

Технические требования (ручная реализация без библиотек):

- Использовать **HTML5 Drag & Drop API** (проще и быстрее) **ИЛИ** события мыши (`mousedown/mousemove/mouseup`) — разрешено любое.
- При перетаскивании:
 - Показывается **полупрозрачная копия** карточки (штатное поведение браузера для Drag & Drop или кастомная)
 - При наведении на колонку она **подсвечивается** (изменение фона или рамки)
- После отпускания:
 - Карточка удаляется из исходной колонки
 - Вставляется в целевую колонку в конец (или в конкретную позицию, если реализован `reorder` — см. бонус)
 - Сохраняется состояние в `localStorage`

Важно:

- Нельзя создать копию карточки при неудачном перетаскивании (например, отпустили вне колонки)
- Должна быть возможность перетаскивать только карточки, а не заголовки колонок

4. Редактирование текста карточки

- **Двойной клик** по тексту карточки (или по всей карточке, но аккуратнее) превращает его в `<input type="text">` с текущим значением.
- Нажатие **Enter** — сохраняет новое значение.
- Потеря фокуса (`blur`) — тоже сохраняет.
- Если новое значение пустое или состоит из пробелов — **удалить карточку** (или оставить старый текст — на усмотрение, но лучше не удалять).
- После сохранения — обновить отображение и `localStorage`.

5. Удаление карточки

- У каждой карточки есть кнопка «» или «X».
- При нажатии:
 - Карточка удаляется без подтверждения (или с confirm — опционально)
 - Состояние сохраняется в localStorage
 - Порядок остальных карточек сохраняется

6. Сохранение состояния (localStorage)

Что сохранять:

- Массив объектов, представляющих все карточки. Каждый объект (например, JSON):

```
{  
  "id": "1623456789012",  
  "text": "Купить молоко",  
  "column": "todo" // возможные значения: "todo", "inprogress", "done"  
}
```
- **Порядок карточек** внутри колонки — сохраняется как порядок элементов в массиве (фильтруем по `column` и берем порядок из массива).

Восстановление при загрузке:

- Если в localStorage есть данные — загрузить их и отрисовать карточки в нужном порядке.
- Если данных нет — пустая доска (или несколько карточек-примеров для наглядности).

7. Дополнительные возможности (бонусные баллы)

Команда может выбрать **любое количество** из списка (время ограничено, но каждая фиша дает дополнительные баллы):

1. **Reorder внутри колонки** — перетаскивание меняет порядок карточек внутри одной колонки (самый сложный бонус).
2. **Счетчики карточек** над каждой колонкой (обновляются при любом изменении).
3. **Анимация** при добавлении/удалении (простое затухание или slide).
4. **Подтверждение удаления** (alert или модальное окно).
5. **Очистка всей доски** (кнопка «Очистить всё» с подтверждением).
6. **Импорт / Экспорт** данных в JSON (кнопка «Сохранить в файл» и «Загрузить из файла»).

Критерии оценки (150 баллов)

Критерий	Баллы	Подробности
Drag & drop между колонками	50	Работает стабильно, без дублирования, визуальная обратная связь
Создание и удаление карточек	25	Поле ввода не пустое, удаление работает, счетчик обновляется
Редактирование двойным кликом	25	Сохраняется на Enter и blur, пустое значение не ломает доску
Сохранение и восстановление (localStorage)	10	Все данные (текст, колонка, порядок) восстанавливаются после reload
Качество интерфейса и CSS	20	Чисто, адаптивно, понятно, нет наложений
Бonusные фиши	20	За каждую реализованную фишу (макс. 4 бонуса по 5 баллов)